

Reviewer Pack — Noncommutative Fourier Spectrum Gap in Read-Twice BP for IP_...

Entry 44a8ac95f70e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 23:44:53 UTC

Noncommutative Fourier Spectrum Gap in Read-Twice BP for IP_2

Entry ID: 44a8ac95f70e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 23:44:53 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative harmonic analysis **Field B** (complexity object): Read-twice branching programs for IP_2

Statement:

For any read-twice BP P with n variables, the spectral norm of its Fourier transform over the symmetric group S_n is $O(\log n)$, but for the IP_2 problem, it is $\Omega(n)$.

Rationale (proposer’s reasoning):

Noncommutative harmonic analysis decomposes BP transition matrices into irreducible representations, revealing structural asymmetries. The spectral norm captures the ‘spread’ of Fourier coefficients, which should be small for structured BPs but large for inherently complex problems like IP_2.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2c4d8514d1dd9540

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=216.1s

5.1 Generated Python source

```
import random
import math

# Helper functions for group theory and linear algebra
def factorial(n):
    if n == 0:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def binomial_coefficient(n, k):
    return factorial(n) // (factorial(k) * factorial(n - k))

def sign(x):
    if x > 0:
        return 1
    elif x < 0:
        return -1
    else:
        return 0

def permutation_to_cycle(perm):
    n = len(perm)
    cycles = []
    visited = [False] * n
    for i in range(n):
        if not visited[i]:
            cycle = []
            j = i
            while not visited[j]:
                visited[j] = True
                cycle.append(j)
                j = perm[j]
            cycles.append(cycle)
    return cycles

def sign_of_permutation(perm):
    cycles = permutation_to_cycle(perm)
```

```

num_transpositions = sum(len(c) - 1 for c in cycles)
return (-1) ** num_transpositions

def young_tableaux(n, k):
    if n == 0:
        return [[]]
    tableaux = []
    for i in range(k + 1):
        if i <= n and (k - i) >= 0:
            subtableaux = young_tableaux(n - i, k - i)
            for subtab in subtableaux:
                new_tab = [row[:] for row in subtab]
                new_tab.append([i] * (n - i))
                tableaux.append(new_tab)
    return tableaux

def hook_length_formula(tableau):
    n = len(tableau)
    hook_lengths = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            hook_lengths[i][j] = (i + 1) + (n - j) - 1
    det = 1
    for row in tableau:
        for x in row:
            det *= hook_lengths[x-1][row.index(x)]
    return abs(det)

def young_symmetric_group_representations(n):
    tableaux = young_tableaux(n, n)
    representations = []
    for tab in tableaux:
        dim = hook_length_formula(tab)
        representation = [[0] * dim for _ in range(dim)]
        for i in range(1, n + 1):
            for j in range(i):
                sign_factor = sign_of_permutation([j+1 if k < i else k+1 for k in range(n)])
                for k in range(dim):
                    representation[k][k] += sign_factor * hook_length_formula(tab[:i-1]) / hook_length_formula(tab)
            representations.append(representation)
    return representations

def matrix_multiplication(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    result = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def gaussian_elimination(A, b):
    m = len(A)
    n = len(A[0])

```

```

augmented_matrix = [row + [b[i]] for i, row in enumerate(A)]
for i in range(n):
    max_row = i
    for j in range(i+1, m):
        if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
            max_row = j
    augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
    pivot = augmented_matrix[i][i]
    for j in range(i, n+1):
        augmented_matrix[i][j] /= pivot
    for j in range(m):
        if j != i:
            factor = augmented_matrix[j][i]
            for k in range(i, n+1):
                augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

x = [0] * n
for i in range(n-1, -1, -1):
    x[i] = augmented_matrix[i][-1]
    for j in range(i+1, n):
        x[i] -= augmented_matrix[i][j] * x[j]
return x

def spectral_norm(matrix):
    m = len(matrix)
    n = len(matrix[0])
    A = matrix
    B = [[sum(A[i][k] * A[k][j] for k in range(n)) for j in range(n)] for i in range(m)]
    eigenvalues = []
    for _ in range(10): # Power iteration method
        x = [random.random() for _ in range(n)]
        x /= sum(x)
        y = matrix_multiplication(A, x)
        y /= sum(y)
        lambda_ = sum(a * b for a, b in zip(x, y))
        eigenvalues.append(lambda_)
    return max(eigenvalues)

def read_twice_bp_transition_matrix(n):
    # Placeholder function to generate a random read-twice BP transition matrix
    return [[random.random() for _ in range(2**n)] for _ in range(2**n)]

def ip_2_transition_matrix(n):
    # Placeholder function to generate the IP_2 transition matrix
    return [[1 if i == j else 0 for j in range(2**n)] for i in range(2**n)]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    if n == 40:
        instances_tested = 20
    else:
        instances_tested = 30

    metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

```

```

for _ in range(instances_tested):
    if random.random() < 0.5: # Read-twice BP
        transition_matrix = read_twice_bp_transition_matrix(n)
    else: # IP_2
        transition_matrix = ip_2_transition_matrix(n)

    representations = young_symmetric_group_representations(n)
    fourier_transforms = []
    for rep in representations:
        transform = gaussian_elimination(rep, transition_matrix[0])
        fourier_transforms.append(transform)

    norm = spectral_norm(fourier_transforms)
    metric_value += norm

    if n == 40 and norm > n: # IP_2 should have  $\Omega(n)$  scaling
        conjecture_holds = False
        counterexample = "IP_2 instance with n=40 has large Fourier norm"

metric_value /= instances_tested

return {
    "metric_name": "Spectral Norm",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3**j - 5 for i in range(5, 8) for j in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing data, preventing evaluation of support fraction or counterexamples. | next: Investigate crash logs for test code or runtime environment issues

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	37054
2	propose	ollama_remote	qwen3:8b	0	0	79118
3	preregistration	ollama_remote	qwen3:8b	0	0	24122
4	novelty	ollama_remote	qwen3:8b	0	0	20745
5	novelty	ollama_remote	qwen3:8b	0	0	17110
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16241
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20763
8	judge	ollama_remote	qwen3:8b	0	0	130917

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 346071 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/44a8ac95f70e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/44a8ac95f70e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/44a8ac95f70e.tar.gz` (if generated)